TestPilot 1C

Установка на Windows, использование и демонстрация

Пошаговое руководство для пользователя
без предварительного опыта работы с сервисом

PowerShell · Python 3.12 · Docker Desktop · Playwright

Версия руководства: 16 октября 2026 года

Репозиторий: https://github.com/ilnurcode/testpilot_1c

Для показа доступен полный процесс на учебном стенде. Живая 1C/OData и автоматический разбор бинарных выгрузок пока не подключены. Полная установка на реальном Windows ещё не проверена.

Содержание

Начните с раздела 1 и выполняйте команды по порядку. Ссылки ниже ведут к соответствующим разделам.

1. Что понадобится

2. Установка Python

3. Установка Git

4. Установка и проверка Docker Desktop

5. Скачивание проекта

6. Установка зависимостей проекта

7. Первый запуск сервиса

8. Пароль и вход в сервис

9. Проверка установки

10. Использование сервиса

11. Демонстрация на 7–10 минут

12. Настройка нейросети — необязательно

13. Остановка и следующий запуск

14. Обновление проекта

15. Устранение ошибок и обращение за помощью

Контрольный список перед передачей или показом

Проверенная база и границы руководства

1. Что понадобится

Инструкция рассчитана на Windows 10/11 x64. Приложения и Playwright запускаются непосредственно в Windows через PowerShell. PostgreSQL работает в Docker Desktop. Docker может использовать WSL2 внутри, но открывать Ubuntu и устанавливать в ней проект не требуется.

Понадобятся:

- компьютер с Windows 10/11 x64;
- интернет для первой установки;
- права администратора для установки системных программ;
- Python 3.12 x64 с Python Launcher;
- Git;
- Docker Desktop с Compose v2 и Linux containers;
- PowerShell 5.1 или 7 и обычный браузер.

Совместимость Docker Desktop с конкретной сборкой Windows проверяйте по его актуальным требованиям. Системные программы устанавливаются один раз. Сам проект обычно запускается без прав администратора.

Все команды ниже, кроме явно указанных исключений, выполняются в PowerShell. Строки вида PS C:\...> показывают приглашение терминала — вводить их не нужно. Копируйте только команды из блоков.

2. Установка Python

1. Откройте официальный сайт: https://www.python.org/downloads/windows/
2. Скачайте установщик Python 3.12 для Windows x64.
3. Запустите установщик. Включите Python Launcher и Add python.exe to PATH, если эти пункты предлагаются.
4. Завершите установку.
5. Откройте новое окно PowerShell через меню «Пуск» и выполните проверку.

```
py -3.12 --version
```

Ожидается ответ вида Python 3.12.x. Если команда не найдена, закройте и заново откройте PowerShell. Если это не помогло, проверьте установку Python Launcher. Нужна именно ветка 3.12, а не просто любая установленная версия Python.

3. Установка Git

1. Откройте https://git-scm.com/download/win
2. Скачайте установщик и установите Git. Стандартные параметры подходят для этой инструкции.
3. Откройте новое окно PowerShell.
4. Проверьте установку.

```
git --version
```

Должна появиться версия Git. Git нужен для скачивания и последующего обновления исходного кода.

4. Установка и проверка Docker Desktop

Docker запускает PostgreSQL — базу данных, в которой сохраняются проекты, исходные файлы, версии и отчёты.

1. Откройте официальную инструкцию: https://docs.docker.com/desktop/setup/install/windows-install/
2. Установите Docker Desktop. Используйте рекомендуемый WSL2 backend и режим Linux containers.

3. Перезагрузите компьютер, если этого требует установщик.
4. Запустите Docker Desktop через меню «Пуск» и завершите первоначальную настройку.
5. Дождитесь статуса работающего движка — например, Engine running.

Отдельную Ubuntu устанавливать и открывать для этого проекта не нужно. При установке Docker может потребоваться включённая аппаратная виртуализация в UEFI/BIOS. Если это рабочий компьютер, за изменением системных настроек обратитесь к администратору.

В PowerShell выполните:

```
docker version
docker compose version
docker info --format '{{.OSType}}'
```

Ожидается: docker version показывает Client и Server; Compose имеет версию v2; последняя команда выводит linux.

Если появляется ошибка failed to connect to the docker API, dockerDesktopLinuxEngine или The system cannot find the file specified, Linux-движок Docker недоступен. Сначала запустите Docker Desktop и дождитесь его готовности.

Если Docker работает в режиме Windows containers, выберите Switch to Linux containers в меню значка Docker возле часов. Если в меню написано Switch to Windows containers, Linux-режим уже выбран.

Если Docker требует обновить WSL, выполните:

```
wsl --update
```

После этого перезапустите Docker Desktop. Для обновления WSL могут потребоваться права администратора. Если Docker по-прежнему не запускается, проверьте статус WSL:

```
wsl --status
```

Не переходите к установке зависимостей проекта, пока Docker не показывает работающий Server и тип linux.

5. Скачивание проекта

Откройте обычный PowerShell. Создайте папку для проектов и скачайте исходники:

```
New-Item -ItemType Directory -Force "$HOME\Projects" | Out-Null
Set-Location "$HOME\Projects"
git clone https://github.com/ilnurcode/testpilot_1c.git
Set-Location testpilot_1c
```

После этого приглашение терминала будет примерно таким: PS C:\Users\ИмяПользователя\Projects\testpilot_1c>

Все последующие команды проекта выполняйте из этой папки. Чтобы вернуться в неё из другого места, используйте:

```
Set-Location "$HOME\Projects\testpilot_1c"
```

Если репозиторий приватный, войдите в GitHub разрешённым способом на своём компьютере; пользователю нужен доступ к репозиторию. Не вставляйте токены в URL или сообщения.

Если папка уже существует, не клонируйте поверх неё. Используйте существующий checkout и раздел «Обновление проекта».

Не размещайте рабочую папку в OneDrive. Не переносите виртуальное окружение .venv из Linux: оно несовместимо с Windows. На другом ПК лучше выполнить свежую установку по этой инструкции.

6. Установка зависимостей проекта

В PowerShell, из папки проекта, разрешите запуск локальных скриптов только в текущем окне:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
```

Если появится запрос, подтвердите изменение. Постоянная политика компьютера не меняется. Если действует корпоративная Group Policy, обратитесь к администратору; обходить её не нужно.

Убедитесь, что Docker Desktop работает, затем выполните:

```
.\scripts\install.ps1
```

Скрипт проверит Docker и Python, создаст .venv, установит зависимости, создаст локальный .env со случайными паролями, скачает совместимый Chromium для Playwright и проверит запуск браузера. При первой установке нужен интернет. Активация .venv вручную не требуется.

Дождитесь завершения команды и возврата приглашения PowerShell. Признак успешного завершения:

```
Installed. Run: .\scripts\start.ps1
```

Существующий .env сохраняется. Не копируйте пустой .env.example поверх созданного файла.

Если вы перенесли .env с Linux и в нём есть TP_CHROMIUM_PATH=/usr/bin/chromium, удалите или исправьте только эту строку. Без явного переопределения Windows использует Chromium Playwright автоматически. Chrome или Edge, установленный в Windows, не заменяет скачивание браузера Playwright.

Если установщик обнаружил несовместимую .venv, сохраните .env, переименуйте только папку .venv и повторите установку.

7. Первый запуск сервиса

Из корня проекта выполните:

```
.\scripts\start.ps1
```

Скрипт проверит Chromium, поднимет PostgreSQL в Docker, применит миграции и запустит три приложения в фоне на 127.0.0.1. При первой загрузке Docker-образа потребуется подождать.

Дождитесь сообщения:

```
Ready: http://localhost:8000
```

Откройте в обычном браузере Windows:

```
http://localhost:8000
```

Если адрес не открылся, попробуйте http://127.0.0.1:8000. Исходные данные демонстрации автоматически не загружаются. Для остановки используйте специальный скрипт из раздела 13.

8. Пароль и вход в сервис

Файл .env создаётся локально установщиком. Чтобы открыть его, выполните из папки проекта:

```
notepad .env
```

Найдите строку:

```
TP_EXPERT_PASSWORD=значение_пароля
```

В браузере используйте логин expert и всё значение после знака =. Не копируйте название TP_EXPERT_PASSWORD= в поле пароля.

Роль expert подходит для демонстрации: она позволяет подтверждать сценарии и запускать учебные проверки. Другие роли — analyst и runner — имеют отдельные пароли в .env.

В .env также хранится пароль базы и могут храниться API-ключи. Не отправляйте файл другим людям, не публикуйте его в GitHub, не показывайте во время трансляции экрана. В Windows храните проект в личной папке пользователя с подходящими NTFS-правами.

На другом ПК установщик создаёт собственные пароли. Не удаляйте .env при уже созданной базе: новый случайный пароль не меняет пароль сохранённой PostgreSQL. Сделайте защищённую копию .env. Исходная папка проекта сама по себе не является резервной копией БД: данные находятся в Docker volume testpilot_pgdata.

9. Проверка установки

При запущенном сервисе выполните:

```
Invoke-RestMethod http://127.0.0.1:8000/health
```

Ответ должен содержать status со значением ok.

Для дополнительной автоматической проверки учебного сценария:

```
. .\scripts\windows-common.ps1
Import-LocalEnv
& .\.venv\Scripts\python.exe scripts\presentation-smoke.py
```

Первая строка начинается с точки и пробела — они нужны. Команды загружают локальные настройки без печати секретов и выполняют учебный сценарий.

Ожидаемый результат содержит passed=2, failed=2, error=0, screenshots=2. Проверка создаёт учебный анализ, выполняет реальные HTTP/Playwright-запросы и проверяет сохранение истории и PNG. Две неуспешные проверки ожидаемы: в учебном приложении намеренно оставлена ошибка расчёта скидки.

10. Использование сервиса

Основной процесс:

1. Создать проект.
2. Загрузить файл разработки.
3. Выбрать полный анализ или сравнение с прошлой версией.
4. Посмотреть изменения и затронутые объекты.
5. Подготовить сценарии проверок.
6. Проверить и подтвердить сценарии и данные.
7. Запустить доступный исполнитель.
8. Изучить результаты, скриншоты и дефекты.

В разделе «Проекты и файлы» создайте проект и сохраняйте в нём версии разработки. Исходные файлы остаются в истории. Повторные одинаковые загрузки распознаются по содержимому. Предыдущую версию можно использовать как базу сравнения.

Формат	Что доступно сейчас
Нормализованный JSON проекта	Хранение и анализ модели
ZIP с configuration.json в корне	Хранение и анализ модели
CFE, CF, EPF, ERF, DT	Хранение оригинала; автоматический разбор пока недоступен

Нормализованный JSON — специальная модель объектов и связей, а не произвольный JSON-файл. На хранение принимаются файлы до 64 МиБ; нормализованный JSON для анализа ограничен 8 МиБ.

Если предыдущей версии нет, анализируется вся доступная модель и представленные в ней зависимости. Если база сравнения есть, вычисляются изменения и область их влияния. Для анализа зависимостей типовой конфигурации они должны присутствовать в модели.

Сохранение бинарного файла ещё не означает, что сервис разобрал его, установил в 1С или протестировал. Для первого знакомства используйте встроенную демонстрацию. Доступный учебный

исполнитель работает только с отдельным синтетическим стендом, а не с произвольным пользовательским проектом.

Разделы интерфейса:

Раздел	Назначение
Обзор	Начало работы и открытие демонстрации
Проекты и файлы	Загрузка, история версий и анализ
Сравнение	Изменения между версиями
Граф и влияние	Связи и объяснения области влияния
Каталог и покрытия	Сохранённые тесты и привязки к объектам
История и дефекты	Сохранённые результаты и найденные расхождения
Расчёт тест-плана	Подбор набора по каталогу и ограничениям
Отчёты	Сохранённые планы и экспорт
Нейросеть и проверки	Предложения, подтверждение и отчёт запуска

11. Демонстрация на 7–10 минут

Перед публичным показом один раз пройдите весь сценарий самостоятельно. Используйте браузер на весь экран, желательно с масштабом 100%. Закройте .env и другие окна с секретами.

Шаг 1. Объясните назначение

Откройте «Обзор». Можно сказать:

«Сервис сохраняет версии разработки, определяет изменения и связанные объекты, помогает подготовить проверки и сохраняет результаты выполнения. Сейчас показываем процесс на учебном стенде».

Сразу обозначьте, что настоящая публикация 1С ещё не подключена.

Шаг 2. Откройте демонстрационный проект

Нажмите «Открыть демонстрацию». Сервис загрузит две учебные версии, сохранит их и выполнит сравнение. Покажите историю файлов в разделе «Проекты и файлы».

Объяснение: «Предыдущая версия остаётся в проекте. Её не нужно каждый раз загружать заново».

Повторное открытие демонстрации не дублирует исходные файлы, но создаёт новый анализ.

Шаг 3. Покажите изменения и влияние

В учебном примере ожидаются 5 изменений и 10 затронутых объектов. Покажите список изменений, затем граф или объясняющие пути связей.

Можно сказать: «Изменённая функция может использоваться другими объектами. Сервис показывает, какие места могли быть затронуты и почему они включены в область проверки».

Область влияния строится по представленным в модели связям. Полнота всех динамических зависимостей настоящей 1С пока не гарантируется.

Шаг 4. Подготовьте сценарии и данные

Нажмите «Подготовить проверки». Выберите «Учебные правила — без нейросети», затем «Сформировать сценарии».

Появятся два предложения: проверка расчёта через HTTP API учебного стенда и проверка расчёта в браузере через Playwright. Покажите обоснование, затронутые объекты, шаги, входные данные и ожидаемые суммы.

Без API-ключа используется явно обозначенный учебный генератор. Не называйте его вызовом нейросети. Корректная формулировка: «Адаптер внешней нейросети подготовлен. В этом показе сценарии поступают из учебных правил. Дальнейшее выполнение и сохранение результатов происходят фактически».

Плановые связи покрытия не являются доказательством полноты всех тестов.

Шаг 5. Подтвердите сценарии

Введите причину подтверждения, например:

```
Проверены входные данные и ожидаемые суммы учебного расчёта скидки.
```

Оставьте выбранными оба сценария и нажмите «Подтвердить выбранные сценарии». Подтверждённые предложения сохраняются как новые тесты в каталоге с указанными экспертом связями покрытия.

Объяснение: «Перед выполнением пользователь проверяет предложенные действия и ожидаемые результаты».

Шаг 6. Выполните проверки

Нажмите «Запустить на учебном стенде» и дождитесь отчёта. Выполняются реальные HTTP-запросы и действия Chromium. Статусы определяются сравнением ожидаемых и фактических значений. Созданные учебные записи удаляются после проверки.

Проверка	Ожидается	Получено	Итог
HTTP: 2 × 100, скидка 10%	180	180	Пройдена
HTTP: 10 × 100, скидка 10%	900	1000	Расхождение
Playwright: 2 × 100, скидка 10%	180	180	Пройдена
Playwright: 10 × 100, скидка 10%	900	1000	Расхождение

Два расхождения ожидаемы: при количестве от 10 учебный стенд намеренно теряет скидку. Это обнаруженная бизнес-ошибка учебного приложения; error=0 означает отсутствие ошибок выполнения в ожидаемом успешном демонстрационном запуске.

Можно сказать: «Обычный заказ проходит, а в оптовом расчёте сервис обнаружил неправильную сумму».

Шаг 7. Покажите доказательства и сохранение

Раскройте «Скриншот выполнения» у браузерной проверки. Покажите ожидаемую и фактическую суммы, статус, настоящий скриншот и записи в разделе «История и дефекты».

Обновите страницу. План, подтверждение, результаты и изображения останутся на месте: они хранятся в PostgreSQL. В Linux проверена их сохранность также после перезапуска приложений.

Завершите показ: «На учебном стенде работает процесс от сравнения версий до найденного дефекта. Следующий интеграционный этап — подключение настоящей 1С и проверка на конкретной конфигурации».

Не говорите «протестировали 1С» или «нейросеть уже вызвана», если использовался учебный режим. HTTP API учебного стенда не является проверенной OData-публикацией 1С.

12. Настройка нейросети — необязательно

Для учебной демонстрации ключ не нужен. Для внешней модели подготовлен OpenAI-совместимый API-адаптер.

Откройте локальный файл:

```
notepad .env
```

Добавьте параметры выбранного провайдера, например:

```
TP_LLM_BASE_URL=https://api.openai.com/v1
TP_LLM_MODEL=gpt-4.1-mini
TP_LLM_API_KEY=ВАШ_КЛЮЧ
```

Реальное значение вписывайте только локально. Не отправляйте его в чат, GitHub и файлы результатов. Сохраните .env и перезапустите сервис:

```
.\scripts\start.ps1
```

В интерфейсе выберите API и подтвердите передачу контекста внешнему провайдеру. Подписка ChatGPT сама по себе не даёт API-ключ. API может требовать отдельную оплату. Наличие ключа не гарантирует доступность выбранной модели.

Фактический вызов реальной модели с ключом пока не проверен. При ошибке API нет скрытого переключения на учебные правила. Подключение модели не добавляет автоматически разбор бинарных файлов или универсальный исполнитель для произвольной 1С. Произвольный программный код, сгенерированный моделью, не запускается.

13. Остановка и следующий запуск

Остановить только приложения, оставив PostgreSQL работающей:

```
.\scripts\stop.ps1
```

Остановить приложения и базу:

```
.\scripts\stop.ps1 -Database
```

Исходные файлы, отчёты, .env и volume testpilot_pgdata сохраняются. Скрипты проверяют принадлежность процессов этому проекту и останавливают также дочерние процессы Windows Python launcher.

После перезагрузки компьютера запустите Docker Desktop, дождитесь готовности движка, откройте PowerShell и выполните:

```
Set-Location "$HOME\Projects\testpilot_1c"
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
.\scripts\start.ps1
```

Повторять установку зависимостей при каждом запуске не требуется. Не используйте docker compose down -v, удаление volume или factory reset Docker для обычной остановки: это может удалить базу.

14. Обновление проекта

Если исходники локально не изменялись:

```
Set-Location "$HOME\Projects\testpilot_1c"
Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned
.\scripts\stop.ps1
git pull --ff-only
.\scripts\install.ps1
.\scripts\start.ps1
```

Install обновит зависимости и браузер, start применит миграции. Локальный .env Git не заменяет. Если Git сообщает о локальных изменениях, сначала сохраните их. Не используйте reset --hard для обхода ошибки.

15. Устранение ошибок и обращение за помощью

Проблема	Что проверить
py или Python 3.12 не найден	Установлены Python 3.12 x64 и Launcher; PowerShell открыт заново
Скрипты запрещены	Выполнена команда RemoteSigned для текущего окна; при корпоративной политике нужен администратор
dockerDesktopLinuxEngine недоступен	Docker Desktop запущен; Linux-движок готов
Chromium не найден	Повторите install.ps1; проверьте TP_CHROMIUM_PATH, не оставляйте Linux-путь /usr/bin/chromium
Несовместимая .venv	Не переносите окружение Linux. Переименуйте только .venv и повторите установку, сохранив .env
Неверный логин или пароль	Используйте expert и пароль из .env именно этой установки
Ошибка пароля PostgreSQL	.env не заменялся после создания базы; восстановите исходный пароль из защищённой копии
Порт занят	Другие приложения не используют 5432, 8000, 8001, 8002; не завершайте чужие процессы наугад
PID record does not match	Скрипт отказался останавливать процесс с неподтверждённой принадлежностью. Проверьте .runtime/windows-*.json и командную строку процесса; это возможно после перемещения папки
Интерфейс не открывается	Дождитесь готовности сервисов, проверьте localhost и /health, затем логи

Для проверки web-сервиса:

```
Invoke-RestMethod http://127.0.0.1:8000/health
```

Посмотреть последние ошибки web-приложения:

```
Get-Content .runtime\windows-web.error.log -Tail 50
```

Посмотреть логи базы:

```
docker compose logs --tail=50 db
```

В папке .runtime также находятся windows-impact.log, windows-impact.error.log, windows-intelligence.log, windows-intelligence.error.log и windows-web.log.

При обращении за помощью приложите текст ошибки, команду и номер шага, на котором она возникла. Не прикладывайте .env, пароли и API-ключи. Перед пересылкой логов проверьте отсутствие секретов и персональных данных. Не публикуйте локальный сервис в интернет для обхода ошибки соединения.

Контрольный список перед передачей или показом

- Docker Desktop показывает работающий Linux-движок.
- Установщик завершился без ошибки; start.ps1 сообщил Ready.
- Страница localhost:8000 открывается, вход expert работает.

- Учебный сценарий пройден заранее: 2 проверки пройдены, 2 ожидаемых расхождения, 0 ошибок выполнения, доступны 2 PNG.
- Файл .env закрыт и не попадает в демонстрацию экрана.
- Понятно обозначены ограничения: нет живой 1С, бинарного парсера и проверенного вызова реальной модели.
- Известна команда остановки: .\scripts\stop.ps1 -Database.

Проверенная база и границы руководства

В облачном Linux прошли 84 pytest с PostgreSQL, сквозные HTTP/Playwright-сценарии и проверка сохранения данных после перезапуска. На PowerShell 7.4.19 для Linux проверены синтаксис скриптов, чтение .env как данных, коды ошибок, пути с пробелами и защита от остановки чужого/повторно использованного PID с контрольными объектами.

Это не заменяет полный запуск Windows: Docker Desktop, настоящий Win32_Process, NTFS и Windows Chromium в облачной проверке недоступны. Windows PowerShell 5.1 отдельно не запускался. Успешная установка на реальном Windows пока не заявляется.

Приложение рассчитано на локальную демонстрацию с общими ролями, а не на публичную промышленную многопользовательскую эксплуатацию.

Полезные ссылки:

- Код: https://github.com/ilnurcode/testpilot_1c
- Актуальная Windows-инструкция: https://github.com/ilnurcode/testpilot_1c/blob/main/docs/windows.md
- Сценарий показа: https://github.com/ilnurcode/testpilot_1c/blob/main/docs/presentation.md